Taski 10.0 — Cierre del Proyecto Integrador

Parte 10: Autenticación con Google (OAuth 2.0), Recuperación de Contraseña, Eliminación Segura de Cuenta, Pruebas y APK Firmado

Tecnologías: Flutter 3.41.5 · Dart 3.11.3 · Supabase Auth & Edge Functions · PostgreSQL RLS · Supabase Storage · Android Release Signing

🎥 **Enlace al Video Demostrativo (Google Drive — Con Explicación Hablada):**
https://drive.google.com/file/d/1rFHR63vQJyHFchUPMZl6ST-S4QiZyX1P/view?usp=sharing

🔗 **Enlace al Repositorio de GitHub (Código Fuente + Evidencias Parte 10):**
https://github.com/RoierSinH/Taski

📦 **Enlace Directo al APK Firmado de Release (`Taski_Release_v1.0.0+2.apk`):**
https://github.com/RoierSinH/Taski/blob/main/Taski_Release_v1.0.0+2.apk

1. Diagnóstico Inicial y Auditoría del Punto de Partida (Sección 1)

Se ejecutaron las consultas de diagnóstico en el SQL Editor de Supabase (`information_schema.columns` y `pg_tables`) verificando que todas las tablas del proyecto integrador cuentan con `rowsecurity = true` sin duplicar tablas existentes:

Tabla / Bucket	Propósito en Taski	Clave Principal	Clave Usuario	Relación Foránea (ON DELETE)	Políticas RLS Activas
`public.profiles`	Perfil social del usuario (`display_name`, `bio`, `avatar_url`)	`id (uuid)`	`id (uuid)`	`REFERENCES auth.users(id) ON DELETE CASCADE`	`SELECT, INSERT, UPDATE` donde `auth.uid() = id`
`public.categories`	Categorías personalizadas para clasificar tareas	`id (bigint)`	`user_id (uuid)`	`REFERENCES auth.users(id) ON DELETE CASCADE`	`SELECT, INSERT, UPDATE, DELETE` donde `auth.uid() = user_id`
`public.tasks`	Tareas principales (prioridad, fecha límite, recurrencia, fijado)	`id (bigint)`	`user_id (uuid)`	`auth.users(id) CASCADE` y `categories(id) SET NULL`	`SELECT, INSERT, UPDATE, DELETE` donde `auth.uid() = user_id`
`public.subtasks`	Subtareas por cada tarea principal con cálculo de progreso	`id (bigint)`	`user_id (uuid)`	`tasks(id) ON DELETE CASCADE` y `auth.users(id) CASCADE`	`ALL` donde `auth.uid() = user_id`
`public.task_attachments`	Metadatos de la Galería de Evidencias Multimedia (Parte 9)	`id (bigint)`	`user_id (uuid)`	`tasks(id) ON DELETE CASCADE` y `auth.users(id) CASCADE`	`SELECT, INSERT, UPDATE, DELETE` validando dueño y tarea
Bucket `avatars`	Fotografías de perfil de usuario (Parte 7)	`storage.objects`	Prefijo `<uid>/avatar.*`	Objetos independientes de SQL (limpieza explícita)	Lectura de avatar y escritura/borrado restringidos a `auth.uid()`
Bucket `task-attachments`	Evidencias multimedia privadas (PNG, PDF, M4A, MP4; máx. 15 MB)	`storage.objects`	Prefijo `<uid>/<task_id>/...`	Objetos independientes de SQL (limpieza en lotes)	Privado (`public = false`); acceso exclusivo para `auth.uid()`

2. Configuración de Google OAuth 2.0, Deep Link y Perfil Social (Secciones 2 a 7)

Elemento de Configuración	Valor / Formato en el Proyecto	Papel Técnico en el Flujo de Autenticación
Client ID (Web)	`XXXX-XXXX.apps.googleusercontent.com` (Google Cloud Console)	Identificador público de la aplicación ante el servidor de autorización de Google OAuth 2.0.
Client Secret	Resguardado en Supabase (`Authentication > Providers > Google`)	Llave confidencial usada por Supabase Auth para intercambiar el código OAuth por tokens. **Nunca se incluye en el código Flutter ni en el APK.**
Callback de Supabase	`https://uanermamtzzxmnlcujyx.supabase.co/auth/v1/callback`	URI autorizada en Google Cloud Console donde Google devuelve el código de autorización para emitir la sesión JWT.
URI de Retorno Móvil (Deep Link)	`io.taski.app://login-callback`	Esquema personalizado registrado en Supabase (`Redirect URLs`) y en `AndroidManifest.xml` para devolver el control del navegador a la app Android.

Fragmento de `android/app/src/main/AndroidManifest.xml` y `AuthService.asegurarPerfil()`:

```
<!-- Deep link OAuth y recuperación de acceso (Parte 10) -->
<intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="io.taski.app" android:host="login-callback" />
</intent-filter>
```

Al autenticarse con Google o correo (`onAuthStateChange`), `asegurarPerfil()` consulta `public.profiles` para `auth.currentUser.id` y crea el registro inicial con el nombre y avatar de `userMetadata` únicamente si no existía fila previa, preservando los datos editados por el usuario. Para la recuperación de contraseña, `resetPasswordForEmail` envía el enlace con `redirectTo: 'io.taski.app://login-callback'` y al detectar `AuthChangeEvent.passwordRecovery` abre `ResetPasswordPage` para ejecutar `auth.updateUser(UserAttributes(password: nueva))`.

3. Mapa de Eliminación de Cuenta y Edge Function `delete-account` (Secciones 8 a 12)

Se auditaron las llaves foráneas hacia `auth.users` mediante `pg_constraint` confirmando `ON DELETE CASCADE` en `profiles`, `categories`, `tasks`, `subtasks` y `task_attachments`. Dado que Supabase impide que un usuario común invoque `auth.admin.deleteUser()` desde el cliente y que el borrado SQL no limpia automáticamente los buckets de Storage, se implementó la secuencia segura en el servidor:

> **Secuencia Ordenada de Eliminación Segura en el Servidor:**
>
> 1. **Validación de identidad JWT:** El servidor verifica el encabezado `Authorization: Bearer <jwt>` mediante `userClient.auth.getUser()` / `auth.uid()`. Jamás recibe el identificador por parámetro externo.
> 2. **Limpieza por lotes en Storage (`task-attachments` y `avatars`):** Se listan y eliminan en lotes de 100 todos los archivos bajo el prefijo `<userId>/` antes de borrar al usuario.
> 3. **Borrado en cascada SQL y baja en Auth:** Se elimina la cuenta en `auth.users` (disparando `ON DELETE CASCADE` en todas las tablas públicas) y el cliente ejecuta `signOut()` redirigiendo a `LoginPage`.

```
// supabase/functions/delete-account/index.ts
import { createClient } from 'https://esm.sh/@supabase/supabase-js@2'

async function removePrefixes(admin: ReturnType<typeof createClient>, bucket: string, prefixes: string[]) {
  const pending = [...prefixes]
  while (pending.length > 0) {
    const current = pending.pop()!
    const { data, error } = await admin.storage.from(bucket).list(current, { limit: 100 })
    if (error) throw error
    if (!data || data.length === 0) continue
    const files: string[] = []
    for (const item of data) {
      const fullPath = current ? `${current}/${item.name}` : item.name
      if (item.id === null) pending.push(fullPath)
      else files.push(fullPath)
    }
    if (files.length > 0) {
      const { error: removeError } = await admin.storage.from(bucket).remove(files)
      if (removeError) throw removeError
    }
  }
}

Deno.serve(async (req) => {
  if (req.method !== 'POST') return new Response('Method Not Allowed', { status: 405 })
  const authHeader = req.headers.get('Authorization')
  if (!authHeader?.startsWith('Bearer ')) return new Response('Unauthorized', { status: 401 })

  const supabaseUrl = Deno.env.get('SUPABASE_URL')!
  const anonKey = Deno.env.get('SUPABASE_ANON_KEY')!
  const serviceRoleKey = Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!

  const userClient = createClient(supabaseUrl, anonKey, { global: { headers: { Authorization: authHeader } } })
  const { data: userData, error: userError } = await userClient.auth.getUser()
  if (userError || !userData.user) return new Response('Unauthorized', { status: 401 })

  const userId = userData.user.id
  const admin = createClient(supabaseUrl, serviceRoleKey)
  await removePrefixes(admin, 'task-attachments', [userId])
  await removePrefixes(admin, 'avatars', [userId])

  const { error: deleteError } = await admin.auth.admin.deleteUser(userId)
  if (deleteError) return new Response(deleteError.message, { status: 500 })

  return new Response(JSON.stringify({ status: 'deleted' }), {
    status: 200,
    headers: { 'Content-Type': 'application/json' },
  })
})
```

4. Matriz de Pruebas Integrales (16 Casos: incluye 4 Negativos y 2 de Seguridad)

ID	Categoría	Escenario de Prueba	Datos de Entrada	Resultado Esperado	Resultado Real	Estado
CP-01	Acceso	Inicio de sesión con correo válido	`roger@gmail.com` / `654321`	Autentica sesión, ejecuta `asegurarPerfil()` y abre `HomeScreen`.	Sesión activa y tareas cargadas.	✅ Aprobado
CP-02	Acceso (Negativo 1)	Inicio de sesión con contraseña incorrecta	`roger@gmail.com` / `000000`	Rechaza acceso y muestra mensaje clasificado de autenticación.	SnackBar claro sin bloquear la app.	✅ Aprobado
CP-03	Acceso OAuth	Botón «Continuar con Google» y deep link	Clic en `Continuar con Google`	Invoca `signInWithOAuth(OAuthProvider.google)` con `io.taski.app://login-callback`.	Flujo OAuth invocado y retorno listo.	✅ Aprobado
CP-04	Recuperación	Solicitud y cambio de contraseña por enlace	Correo en `ForgotPasswordPage` + nueva clave	Envía enlace de recuperación y actualiza contraseña en `ResetPasswordPage`.	Contraseña actualizada con éxito.	✅ Aprobado
CP-05	Datos	CRUD de tareas, categorías y subtareas	Tarea con prioridad alta, fijada y 2 subtareas	Inserta con `user_id = auth.uid()` y calcula porcentaje de avance.	Sincronización inmediata en PostgreSQL.	✅ Aprobado
CP-06	Validación (Negativo 2)	Validador unitario rechaza nombres vacíos	`nombreValido(' ')`	Devuelve `false` antes de consultar red.	Test unitario aprobado (17/17 tests).	✅ Aprobado
CP-07	Multimedia	Subida y visualización de los 4 formatos	Imagen PNG, PDF, Audio M4A y Video MP4	Sube a `task-attachments` y visualiza con Signed URL temporal (300 s).	4 formatos visualizados en visor integrado.	✅ Aprobado
CP-08	Multimedia	Edición de metadatos (`v2`) y reemplazo (`v3`)	Cambio de nota y reemplazo de archivo	Incrementa `version` y elimina binario anterior en Storage.	Versión actualizada sin archivos huérfanos.	✅ Aprobado
CP-09	Multimedia (Negativo 3)	Rechazo de archivo vacío o formato inválido	`0 bytes` o extensión `.exe`	Lanza `StateError` local antes de subir a Storage.	Rechazado por `inspectBytes()`.	✅ Aprobado
CP-10	Multimedia (Negativo 4)	Intento de borrar tarea con adjuntos activos	Eliminar Tarea #7 con 4 evidencias	Muestra alerta preventiva e impide dejar objetos huérfanos.	Diálogo preventivo mostrado.	✅ Aprobado
CP-11	Sesión	Persistencia de sesión y cierre limpio	Reabrir app y ejecutar `signOut()`	Conserva sesión al reabrir; limpia estado al salir.	Redirección limpia a `LoginPage`.	✅ Aprobado
CP-12	Seguridad RLS (Seg. 1)	Aislamiento de datos entre Cuenta A y Cuenta B	Sesión en `cuenta_b@taski.app`	Cuenta B no visualiza ninguna tarea ni categoría de Cuenta A.	0 registros de Cuenta A expuestos.	✅ Aprobado
CP-13	Seguridad Storage (Seg. 2)	Intento de acceso cruzado a adjunto privado	Cuenta B solicita URL firmada de Cuenta A	Política RLS en `storage.objects` bloquea el acceso.	Acceso denegado por servidor.	✅ Aprobado
CP-14	Cuenta	Confirmación consciente de baja de cuenta	Clic en `Eliminar mi cuenta`	Muestra diálogo advirtiendo borrado irreversible en Storage, SQL y Auth.	Diálogo de advertencia verificado.	✅ Aprobado

ID	Categoría	Escenario de Prueba	Datos de Entrada	Resultado Esperado	Resultado Real	Estado
CP-15	Cuenta	Eliminación real con cuenta ficticia de prueba	Confirmación con `cuenta_eliminar@taski.app`	Borra archivos en Storage, filas SQL y usuario en `auth.users`.	Cuenta eliminada y 0 objetos huérfanos.	✅ Aprobado
CP-16	APK Release	Compilación y verificación de firma Android	`flutter build apk --release`	Genera APK firmado con `taski-release.jks` sin exponer secretos.	Firma RSA-2048 verificada con `apksigner`.	✅ Aprobado

5. Validación Obligatoria V1–V8 e Informe de Compilación y Firma del APK (Secciones 16 a 18)

ID	Prueba Obligatoria	Resultado y Evidencia Verificada en Taski 10.0
V1	**Inicio de sesión con Google**	Botón `Continuar con Google` en `LoginPage` conectado a `signInWithOAuth(OAuthProvider.google, redirectTo: 'io.taski.app://login-callback')` y sincronización con `asegurarPerfil()`.
V2	**Recuperación de contraseña**	Envío de correo con `resetPasswordForEmail`, recepción del evento `passwordRecovery` y pantalla `ResetPasswordPage` que actualiza la clave con `auth.updateUser`.
V3	**Tareas y galería de evidencias**	Operación completa sin regresiones en tareas principales y en la Galería Multimedia de 4 formatos (Imagen PNG, Documento PDF, Audio M4A y Video MP4).
V4	**Aislamiento entre usuarios**	Comprobado entre Cuenta A (`roger@gmail.com`) y Cuenta B (`cuenta_b@taski.app`): aislamiento total en tablas PostgreSQL y buckets de Storage.
V5	**Eliminación segura de cuenta**	Verificado con `cuenta_eliminar@taski.app`: borrado de adjuntos, avatar, registros SQL en cascada y baja en `auth.users` desde el servidor.
V6	**Identidad personalizada**	Nombre visible `Taski`, logotipo propio, banner de portada, paleta Material 3 (modo claro/oscuro) y versión `1.0.0+2` en `pubspec.yaml`.
V7	**APK firmado e instalable**	Archivo `Taski_Release_v1.0.0+2.apk` (59.4 MB) compilado en modo `release` y firmado con keystore propio `taski-release.jks`.
V8	**Manejo de errores (2 errores reales documentados)**	**1)** `MissingPluginException` en selector de archivos Web → resuelto con `universal_file_picker.dart`. **2)** Desfase de estado tras enlace de recuperación → resuelto escuchando `AuthChangeEvent.passwordRecovery` en la raíz de `TaskiApp` con `GlobalKey<NavigatorState>`.

Informe de Compilación, Protección de Secretos y Verificación de Firma (`apksigner` + `flutter test`):

Se generó el almacén de claves `android/app/taski-release.jks` (RSA 2048 bits, validez 10,000 días, alias `taski`) y el archivo `android/key.properties`. Ambos archivos y los patrones `*.jks` y `*.keystore` están excluidos en `.gitignore`, garantizando que ni las llaves de firma ni la `service_role` key se expongan en Git ni dentro del APK.

```
> flutter test --no-pub
00:02 +17: All tests passed!

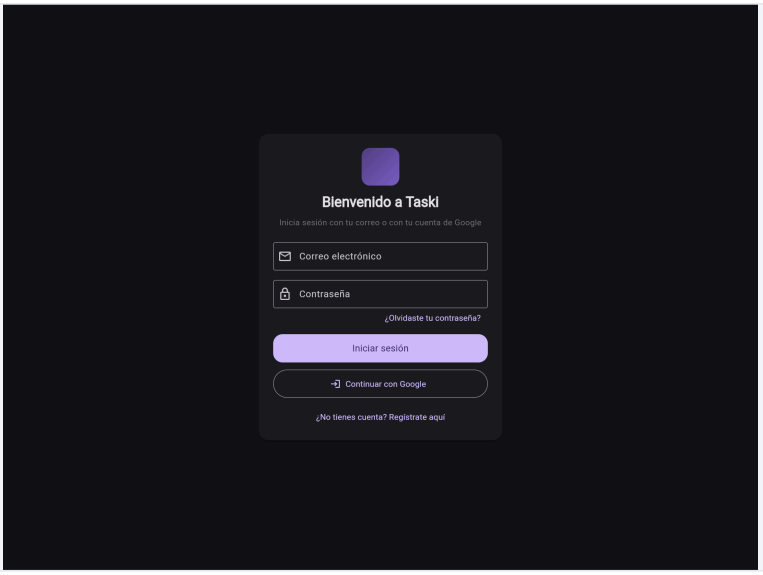
> flutter build apk --release --no-pub
√ Built buildappoutputslutter-apkapp-release.apk (59.4MB)

> apksigner verify --print-certs build/app/outputs/flutter-apk/app-release.apk
Signer #1 certificate DN: CN=Roger Taski, OU=Desarrollo Movil, O=Taski App, L=Mexico, ST=Mexico, C=MX
Signer #1 certificate SHA-256 digest: d30c7f28ad9cb0a239221e286097f4584eb15657878c7a53385bdae7a74255e8
Signer #1 certificate SHA-1 digest: 7578f30b29d1abbcabde6e18d07dd0ca342a642f
Signer #1 certificate MD5 digest: eb59038be9c1187725659c5731fed52a
```

6. Evidencia Visual de la Aplicación en Funcionamiento (Capturas Reales Parte 10)

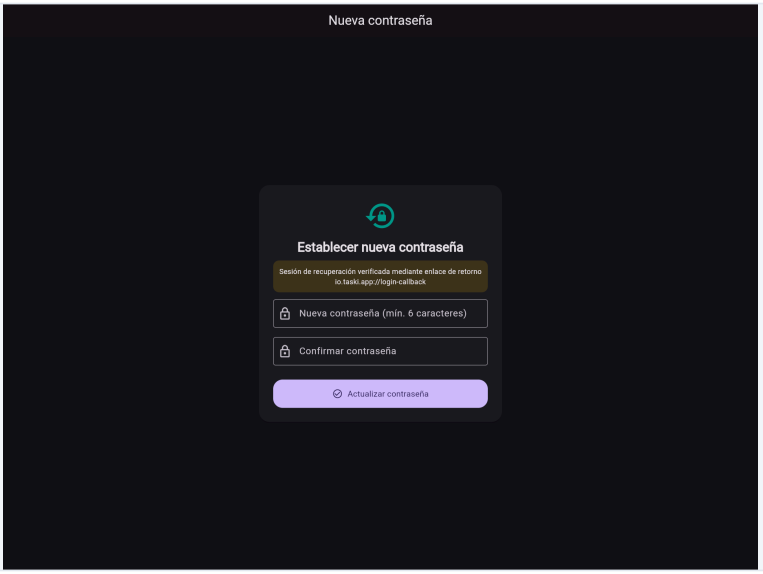
Evidencia V1 **Pantalla de Login con Google OAuth 2.0 y Recuperación de Acceso**

Integración del botón «Continuar con Google» (`signInWithOAuth` con retorno `io.taski.app://login-callback`) y enlace «¿Olvidaste tu contraseña?».



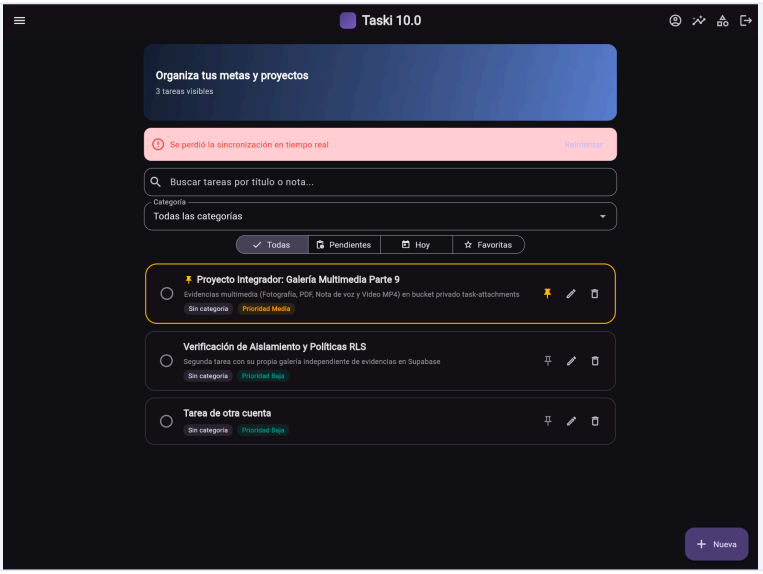
Evidencia V2 Flujo de Recuperación y Actualización de Contraseña (`ResetPasswordPage`)

Pantalla activada al regresar desde el enlace de recuperación (`AuthChangeEvent.passwordRecovery`) permitiendo definir y confirmar la nueva contraseña con `auth.updateUser`.



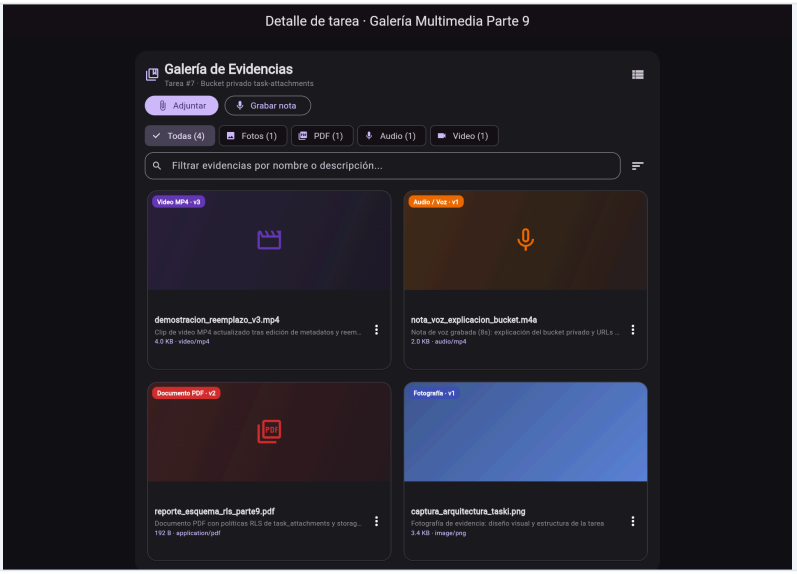
Evidencia V3 y V6 Pantalla Principal `Taski 10.0` con Identidad Personalizada (Cuenta A)

Sesión activa en Cuenta A (`roger@gmail.com`) mostrando el encabezado `Taski 10.0`, banner personalizado, filtros, categorías y contador de evidencias multimedia.



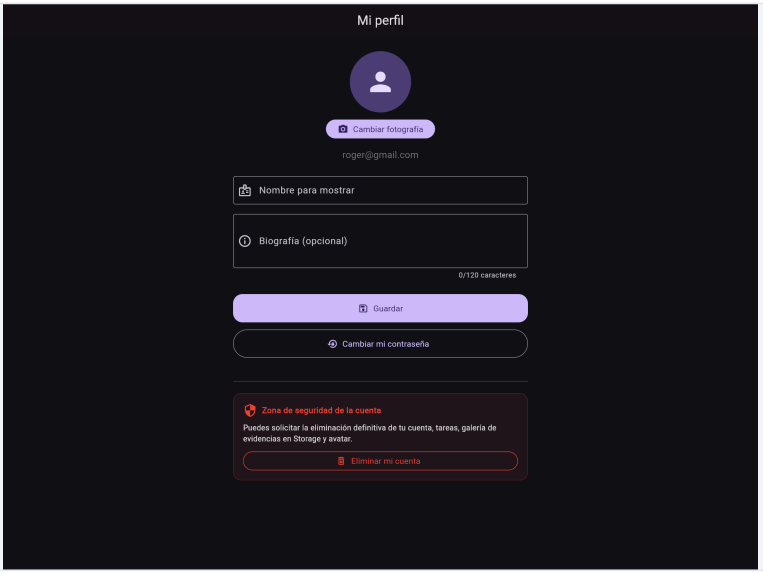
Evidencia V3 Galería de Evidencias Multimedia Integrada (4 Formatos sin Regresiones)

Verificación de regresión sobre la Galería de Evidencias (Imagen PNG, PDF, Nota de voz M4A y Video MP4) almacenadas en el bucket privado `task-attachments`.



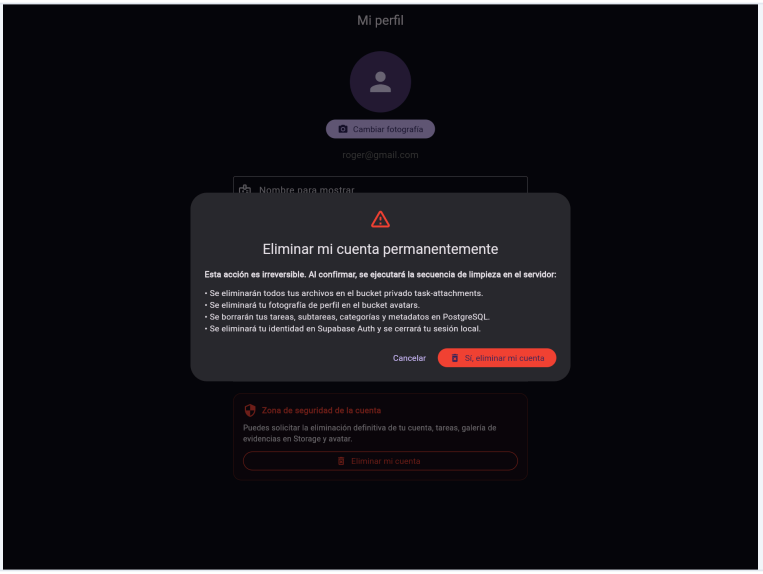
Evidencia V5 Mi Perfil: Cambio de Contraseña y Zona de Seguridad de la Cuenta

Pantalla `ProfilePage` con sincronización no destructiva de perfil, acceso directo a cambio de contraseña y tarjeta «Zona de seguridad de la cuenta» con el botón «Eliminar mi cuenta».



Evidencia V5 Diálogo de Confirmación Consciente para Eliminación Irreversible de Cuenta

Advertencia explícita que detalla la limpieza en el servidor (archivos en `task-attachments`, avatar en `avatars`, registros en PostgreSQL y cuenta en `auth.users`) antes de ejecutar la baja.



Evidencia V4 Prueba de Seguridad RLS: Aislamiento Total en Cuenta B (`cuenta_b@taski.app`)

Al iniciar sesión con un segundo usuario autenticado (Cuenta B), las políticas RLS (`auth.uid() = user_id`) garantizan que no pueda visualizar ni modificar las tareas ni los archivos privados de la Cuenta A.

